Understanding Microservices Architecture

Lesson 1.1: What are Microservices?

Tarun sharma

21/01/2026

What Are Microservices?

Small, Independent Services

Each microservice is focused on a specific task.

Own Process and Data

Services run separately and manage their own databases.

API-Based Communication

Services interact through well-defined APIs.

Monolith vs Microservices

Monolithic Applications: One Big Codebase

All features live in a single, unified project.

Microservices: Specialized, Independent Services

Each service focuses on a specific function.

Independent Deployment & Scaling

Services can be updated or scaled without affecting others.

Why Microservices Matter

Monolithic Growth Challenges

Large codebases slow development and complicate management.

Accelerated & Safer Deployments

Microservices allow rapid updates with reduced deployment risk.

Empowered, Independent Teams

Teams can innovate using diverse technologies and work autonomously.

Real-World Microservices

E-commerce Microservices

Separate services for product, cart, payment, and user management.

Streaming Platform Microservices

Distinct services for user profiles, content delivery, recommendations, and billing.

Banking Application Microservices

Dedicated services for accounts, transactions, fraud detection, and notifications.

How Services Communicate

Synchronous Communication

Direct requests using REST or gRPC APIs

Asynchronous Messaging

Services exchange events via message brokers

API Gateway Role

Centralizes and manages service interactions

Microservices in Practice

Bookstore split into specialized services

User, book, order, and payment services operate independently

Each service manages its own database

Data ownership ensures isolation and flexibility

Independent deployment with containers

Docker streamlines deployment and scaling for each service

Key Takeaways

Independent, Specialized Services

Each microservice focuses on a single function and operates autonomously.

Scalability and Flexibility

Services can be scaled and deployed individually to meet demand.

Solves Monolith Challenges

Addresses issues like slow development and risky deployments in large apps.

Added Complexity

Requires careful management of service communication and data.